Outlook

[WordPress Plugin Directory] Review in Progress: EBQ SEO

From WordPress.org Plugin Directory <plugins@wordpress.org>
Date Wed 5/20/2026 3:57 PM
To malihaider55123 <ali@ebq.io>

👋 malihaider55123 - Let’s improve your plugin!

Thank you for submitting your plugin, "EBQ SEO".

Our volunteer reviewers, tools, and/or AI aids identified **issues in your plugin that require your attention**.

We’ve **pended** your submission to give you a chance to review and fix these common issues.

This team handles approximately 1,500 plugin reviews each week. That's a lot. To make the most of this process, **please do your part and help us help you**; otherwise, your plugin won't be approved.

🤖 *Please note that this message was generated using a combination of humans, algorithms, and AI in varying proportions. It may not have been reviewed by a human. All AI outputs are marked with the ✨ emoji. Pay attention to it, it's quite accurate.*

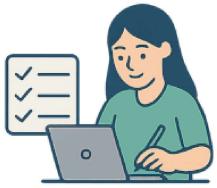
Who are we?

We are a group of volunteers who **help you identify common issues** so that **you can make your plugin more secure, compatible, reliable and compliant** with the guidelines.

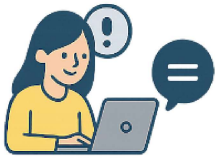
For consistency and better communication, your plugin review will be assigned to a single volunteer who will assist you throughout the entire review. However, response times may vary depending on how much time the volunteer is able to contribute to the team and if whether they need to consult something with the rest of the team.

The review process

Please **read this email in full and check each issue**, as well as the links to the documentation and the provided examples. Also, search for any other similar occurrences of the same issue that are not explicitly mentioned in the email.
Make sure you understand the issues so that you can incorporate them into your existing skillset.



If you decide to continue with the review process, **you must fix any issues, test your plugin, upload a corrected version** and then reply to this email. In case of any doubt, please fix everything else and ask your questions alongside the update.



Your plugin is **manually checked by a volunteer** who sends you the remaining identified issues in the plugin.
We will be devoting our time to reviewing your plugin, we ask that you honor this by following the instructions.
Note: Volunteers are not your QA team. They are here to help you identify and understand issues so that you can improve and maintain your plugin in the future. Fixing the issues is your responsibility.

If there are no further issues, the plugin will be approved 🎉

Be brief and direct in your reply (please, avoid copy-pasting bloated AI responses, our AI is quite brief), be patient and make sure you have **addressed all the issues and tested your plugin before responding**.
It is disheartening to receive an updated plugin only to find that only a few issues have been resolved, or that it causes a fatal error upon activation.

When not making adequate progress in your review, **it will be delayed and eventually rejected**, for the sake of volunteers devoting their time and other plugin authors who correctly follow the review process. Once rejected under these circumstances, it will not be reviewed again.

Each volunteer can review up to 400 plugins per week, and they love to have life besides reviewing plugins, so please make things easier for us.

Understanding the Review Queue

When you reply, your plugin enters the review queue again. Fewer review cycles mean quicker approval, while multiple reviews can extend the process to weeks or months.
Tip: Carefully fix all issues and test your plugin before resubmitting to speed up approval.

This process is designed to help you improve your plugin while making the review experience faster and more efficient for everyone.

Have you read the guidelines and this plugin complies with them?

Upon submitting your plugin, you agreed and confirmed that it complies with the WordPress.org Plugin Directory Guidelines, which apply to all plugins in the directory.

Our automated tools have detected patterns that may require a closer look regarding compliance with certain guidelines. We will verify this during our manual review, but it's best to address any potential issues beforehand. In particular, please pay attention to the following:

- Any included code **must be GPL-compatible**. This means, among others, that the users have to receive the four essential freedoms: (0) to run the program, (1) to study and change the program in source code form, (2) to redistribute exact copies, and (3) to distribute modified versions. *(Guidelines 1 & 4)*
- Plugins **must not restrict or lock functionality**. *(Guidelines 1, 5, & 9 — see clarification on SaaS in Guideline 6)*
- Plugins **should not hijack the admin dashboard**. Upgrade prompts, notices, alerts, and the like must be limited in scope and used with moderation. *(Guideline 11)*

Please check it, and if you think everything is fine, do not worry. Our tools are very thorough and may highlight different things as potential issues.

Have you checked for common technical issues?

Please ensure that your plugin adheres to best practices, including the following:

🔴 Trialware and Locked Features

Please review your plugin to ensure that it **does not include any locked or restricted built-in functionality**. This is not permitted under the WordPress.org Plugin Directory Guidelines you agreed to when submitting the plugin.

❌ Guideline 5 – Trialware

Plugins must be fully functional. You may not:

- Lock, disable or limit built-in features behind a license key, trial period, usage limit, time, quota or any other kind of intended restriction.

Even if the locked feature is present in the code "just in case the user upgrades," it's still **not allowed**. Your plugin may point out which features are available through a separated plugin, but that's it. All plugin code hosted on WordPress.org must be **free and fully functional**.

🌐 Guideline 6 – Serviceware

Plugins may connect to a **legitimate external service to perform certain functionality**, provided:

- The service performs actual processing on external servers.
- The functionality provided cannot be done locally by the plugin.
- The service is clearly documented in your readme, including **Terms of Use** and **Privacy Policy** links.

For example: a "Spam checker" plugin that connects to a external service to check for spam (and thus uses it to provide that functionality) is generally acceptable. A plugin that simply checks a license key to unlock local features is not.

✅ **Ask yourself:**

- Does any function **only work after a license check or payment**?
- Is any functionality in the plugin code **disabled or limited** until it's unlocked?
- Are there any limitations on the plugin **after a certain amount of time or usage**?

After excluding functionalities provided by legitimate external services, if the answer is ***yes*** to any of the above, the plugin **does not comply**.

🔧 **How to fix it:**

- **Remove all license checks** or other mechanisms that control access to features built in in the plugin code.
- **Remove or fully enable** any built in features that are currently locked or limited.
- Make sure external services are compliant and clearly documented.

ℹ️ **Important clarification:**

WordPress.org is **not a marketplace**. It's a repository for **free, fully functional, GPL-compliant plugins**.

If you are not offering a service and want to offer additional features through a paid version, that code must be:

- **Hosted elsewhere** (e.g., your own website).
- **Not included** in the plugin hosted on WordPress.org.
- **GPL compliant**: Do **not** include any mechanisms that would prevent a plug-in from being used after a license has been checked.

✨ `Remote feature flags intentionally disable built-in functionality, including the plugin's local redirects manager and posts-list column, even though those features are implemented in the code and advertised as toggleable per website`

⚠️ The AI has highlighted the most apparent issues. There may be additional concerns not explicitly mentioned. You **must** read and comprehend the guidelines and **review the entire code thoroughly to ensure that there are no other issues**.
❗ If more issues of the same nature are found in the following review, **this plugin will be rejected and will not be reviewed again**. Ensure full compliance with the guidelines to avoid rejection.

🔴 Use wp_enqueue commands

ℹ️ **Why it matters**: Because of performance and compatibility, please make use of the built in functions for including static and dynamic JS and/or CSS.

🔍 **Identify JS and CSS outputs**: Look for any `<script>` or `<style>` HTML tags in your plugin. In the majority of cases you could enqueue them.

🛠 **Fix it**: Make use of the specific function for enqueue them:

Type of code	Functions
Static JS	`wp_register_script()`, `wp_enqueue_script()`, `admin_enqueue_scripts()`
Inline JS	`wp_add_inline_script()`
Static CSS	`wp_register_style()`, `wp_enqueue_style()`
Inline CSS	`wp_add_inline_style()`

👉 In the public pages you can enqueue them using the hook `wp_enqueue_scripts()`.
👉 In the admin pages you can enqueue them using the hook `admin_enqueue_scripts()`. You can also use `admin_print_scripts()` and `admin_print_styles()`.
👉 As of WordPress 6.3, you can easily pass attributes like defer or async, as of WordPress 5.7, you can pass other attributes by using functions and filters.

Example:

```
function ebqse_enqueue_script() {
    wp_enqueue_script( 'ebqse_js', plugins_url( 'inc/main.js', __FILE__ ),
array(), EBQSE_VERSION, true );
}
add_action( 'wp_enqueue_scripts', 'ebqse_enqueue_script' );
```

Your JS/CSS is now **enqueued**!

Possible cases from your plugin include:

```
includes/class-ebq-settings.php:762 <style>
includes/class-ebq-schema-shortcode.php:65 return '<style id="ebq-schema-card-
css">'
includes/class-ebq-meta-box.php:193 <style>
includes/class-ebq-settings.php:785 <script>
includes/class-ebq-redirects-admin.php:235 <script>
includes/class-ebq-seo-fields-meta-box.php:172 '<script>window.ebqClassicMeta =
%s;</script>',
```

🔴 Nonces and User Permissions Before Processing Requests

ℹ️ **Why it matters**: Nonces and permissions checks ensure that the request comes from a trusted source, protecting against security threats like CSRF (Cross-Site Request Forgery) attacks.
Please check the official WordPress docs on nonces and this article for details — this is a quick summary.

🔍 **Spot it**: Look for functions that interact with `$_GET`, `$_POST`, `$_REQUEST` and perform actions

triggered by the user/browser that modify data or perform sensitive actions (e.g., privileged actions or data manipulation). For such actions, you should always check for a nonce.
Additionally, verify user permissions if the action is restricted to certain roles (e.g., admin, editor).
When in doubt, play it safe: always check.

🛠 Fix it

1. Create a nonce (`wp_nonce_field()`, `wp_nonce_url()`, `wp_create_nonce()`). If you need to pass the nonce to JavaScript you can make use of `wp_localize_script()`
2. Pass it with the request.
3. Check the nonce (`check_admin_referer()`, `check_ajax_referer()`, `wp_verify_nonce()`), and permissions if applicable (`current_user_can()`).

Example (nonce check):

```
function ebqse_save_email(){
    if ( !isset( $_POST['ebqse_nonce'] ) || !wp_verify_nonce( sanitize_text_field(
wp_unslash( $_POST['ebqse_nonce'] ) ), 'ebqse_save_email_action' ) ) {
        wp_send_json_error( 'Invalid nonce.' );
    }

    if ( !current_user_can( 'edit_posts' ) ) {
        wp_send_json_error( 'Insufficient permissions.' );
    }

    if ( isset( $_POST['post_id'] ) && isset( $_POST['price'] ) ) {
        update_post_meta( absint( $_POST['post_id'] ), 'price', floatval(
$_POST['price'] ) );
    }
}
add_action( 'wp_ajax_ebqse_save_email', 'ebqse_save_email' );
```

⚠️ Without nonce and permissions checks, anyone could call this AJAX endpoint and change the price data for any post - risky!

🔴 Proper escaping of outputs

ℹ️ **Why it matters**: Escaping prevents common security issues like XSS (Cross-Site Scripting) and avoids breaking the HTML structure.
Please check the official WordPress docs on escaping for details — this is a quick summary.

🔍 **Identify unescaped outputs**: Check any output (`echo`, `print`, `printf`, etc.) outputting a variable (like `$title`, `$key`) or function result (like `get_url()`)
If the value is output without escaping, that's a potential risk! 🦹

🛠 **Fix it**: Always wrap any output with the proper escaping function depending on the context, like:

Context	Function
URLs	`esc_url()`
HTML attributes	`esc_attr()`
Text inside HTML tags	`esc_html()`

Context	Function
Raw HTML (careful!)	`wp_kses()`, `wp_kses_post()`

Refer to the official WordPress documentation for a complete list of escaping functions.
👉 Use the most restrictive function that fits the context.
👉 Escaping should be applied as late as possible, ideally right before output.

Example:

```
echo '<a href="' . esc_url( $profile_link ) . '" class="' . esc_attr( $username )
. '">' . esc_html( $display_name ) . '</a>';
```

Your output is now **escaped**!

Examples from your code:

```
includes/class-ebq-seo-fields-meta-box.php:173 wp_json_encode($meta_seed,
JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES)
# ✨ JSON is printed directly inside a script tag with JSON_UNESCAPED_SLASHES, so
stored meta containing </script> is not safely escaped for HTML script context
```

Note: when you need to echo a JSON, it's better to make use of the function `wp_json_encode`, also, make sure you are not avoiding escaping with the options passed on the second parameter.

```
    echo wp_json_encode($array_or_object);
```

Example(s) from your plugin:

```
includes/class-ebq-seo-fields-meta-box.php:173 wp_json_encode($meta_seed,
JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES)
```

✔️ You can check this using Plugin Check.

🔴 Use Prefixes for declarations, globals and stored data

ℹ️ **Why it matters**: Prefixing avoid naming collisions with other themes, plugins, or WordPress core functions.
A prefix is a string placed in front of a name to avoid collisions. **It must be at least 4 characters long, feel distinct and unique to the plugin (do not use common words)**, and be separated by an underscore or dash.
Please check the official WordPress docs on avoiding name collisions.

🔍 **Identify not prefixed names**: Look for any name that is used in a place where it can create a collision.

Type of element	Affected elements
Declarations	Functions, classes, etc (if not under a namespace)
Globals	Global variables, namespaces, `define()`.

Type of element	Affected elements
Data storage	`update_option()`, `set_transient()`, `update_post_meta()`, etc.
WordPress declarations	`add_shortcode()`, `register_post_type()`, `add_menu_page()`, `wp_register_script()`, `wp_localize_script()`, `add_action( 'wp_ajax_...' )`, etc.

If the defined name for that is not prefixed, that's a potential issue! 🕵️

🛠 **Fix it**: Always prefix those names, for example if your plugin is called "EBQ SEO" then you could use names like these:

- function ebqse_save_post(){ ... }
- class EBQSE_Admin { ... }
- update_option('ebqse_options', $options);
- register_setting('ebqse_settings', 'ebqse_user_id', ...);
- define('EBQSE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global $ebqse_options;
- add_action('wp_ajax_ebqse_save_data', ...);
- namespace malihaider55123\ebqseo;

Other details

We've detected some other details that you may want to check.

The URL(s) declared in your plugin seems to be invalid or does not work.

From your plugin:

Terms/Privacy URL: https://ebq.io/privacy - readme.txt - This URL replies us with a 404 HTTP code, meaning that it does not exists or it is not a public URL.

You haven't added yourself to the "Contributors" list for this plugin.

In your readme file, the "Contributors" parameter is a case-sensitive, comma-separated list of all WordPress.org usernames that have contributed to the code.

Your username is not in this list, you need to add yourself if you want to appear listed as a contributor to this plugin.

If you don't want to appear that's fine, this is not mandatory, we're informing you just in case.

Analysis result:

WARNING: None of the listed contributors "ebq" is the WordPress.org username of the owner of the plugin "malihaider55123".

The source code of your plugin should be publicly accessible.

By submitting a plugin to the directory, you agree to follow the guidelines:

- Guideline **#1** requires that your plugin be compatible with the GPL license.

- Guideline **#4** requires that plugin code cannot be obfuscated and must provide a public, maintained access to their source code and any build tools.

In practice, this means that **the source code of your plugin must be public and available**.

Open source relies on the ability for others to review, study, modify, and even fork the code. For that reason, the source used to generate any compiled or minified assets must be available in an easy-to-find location.

Our automated tools detected compressed or minified files generated by build tools (such as npm, webpack, etc.), but were unable to match them to a non-compiled (human-readable and modifiable) version of the same code.

Please review the reported cases and ensure that the corresponding source code is available and publicly accessible. Our volunteers will verify this during future manual reviews.

- **For your own code:** You may either include the source files within the plugin or provide a link to a publicly accessible source repository. In either case, please document where the source code can be found in the plugin's `readme` file.
- **For third-party libraries:** Ensure they are properly documented. Most libraries include a header comment with the library name, version, and repository URL. If this information is not present in the file itself, please document it in the `readme` file.

We also strongly recommend documenting any build tools and steps required to regenerate the compiled files. This helps future developers understand and maintain the plugin.

From your plugin:

```
build/chatbot.js:7  ...n.o(e,a)&&e[a]&&e[a][0](),e[a]=0;return
n.O(_)},o=globalThis.webpackChunkebq_seo_wp=globalThis.webpackChunkebq_seo_wp||
[];o.forEach(t.bind(null,0)),o.push=t.bind(null,o.push.bind(o))})();var
r=n.O(voi...
# ✨ Generated build/chatbot.js bundle is present with asset metadata, but no
matching source tree or public source repository is provided.
build/805.js:1  ..."use strict";
(globalThis.webpackChunkebq_seo_wp=globalThis.webpackChunkebq_seo_wp||
[]).push([[805],{805(e,t,s){s.d(t,{registerSlashCommand:()=>p});var
n=s(87),a=s(723),o=s(619),i=s(491),c=s(143),r=s(4...
# ✨ Generated webpack split chunk 805.js is shipped without bundled human-
readable source or any public source repository reference.
build/setup.js:2  ...er SEO plugin, configure defaults","ebq-seo")))))}function
h(){const e=new
URLSearchParams(window.location.search),t=e.get("step"),n=e.get("ebq_billing"),s="
success"===n||"cancelled"===n?4:"pricing"==...
# ✨ Generated build/setup.js bundle is present with asset metadata, but no
matching source tree or public source repository is provided.
build/post-bulk-ai.js:3  ...r.o(e,i)&&e[i]&&e[i][0](),e[i]=0;return
r.O(d)},s=globalThis.webpackChunkebq_seo_wp=globalThis.webpackChunkebq_seo_wp||
[];s.forEach(t.bind(null,0)),s.push=t.bind(null,s.push.bind(s))})();var
n=r.O(voi...
# ✨ Generated build/post-bulk-ai.js bundle is present with asset metadata, but
no matching source tree or public source repository is provided.
build/sidebar.js:1  ...(()=>{"use strict";var e={n:t=>{var s=t&&t.__esModule?
()=>t.default:()=>t;return e.d(s,{a:s}),s},d:(t,s)=>{for(var a in
```

```
s)e.o(s,a)&&!e.o(t,a)&&Object.defineProperty(t,a,{enumerable:!0,get:s[a]})},o:
(e...
# ✨ Generated build/sidebar.js bundle is present with asset metadata, but no
matching source tree or public source repository is provided.
build/meta-box-hydrate.js:1  ...(()=>{"use strict";var e={n:t=>{var
o=t&&t.__esModule?()=>t.default:()=>t;return e.d(o,{a:o}),o},d:(t,o)=>{for(var n
in o)e.o(o,n)&&!e.o(t,n)&&Object.defineProperty(t,n,{enumerable:!0,get:o[n]})},o:
(e...
# ✨ Generated build/meta-box-hydrate.js bundle is present with asset metadata,
but no matching source tree or public source repository is provided.
build/459.js:1  ..."use strict";
(globalThis.webpackChunkebq_seo_wp=globalThis.webpackChunkebq_seo_wp||
[]).push([[459],{459(e,t,r){r.d(t,{registerGhostText:()=>l});var
n=r(87),s=r(619),c=r(491),i=r(143),o=r(974),a=r(481)...
# ✨ Generated webpack split chunk 459.js is shipped without bundled human-
readable source or any public source repository reference.
build/hq.js:1  ...(()=>{"use strict";var e={650(e,s,a){a.d(s,{$M:()=>l,L3:
()=>r,jI:()=>d});var t=a(455),n=a.n(t);const i="undefined"!=typeof
window&&window.ebqHqConfig||{};n().use(n().createNonceMiddleware(i.nonce||"")...
# ✨ Generated build/hq.js bundle is present with asset metadata, but no matching
source tree or public source repository is provided.
```

... out of a total of 23 incidences.

Check `permission_callback` in REST API Route

When using `register_rest_route()` or `wp_register_ability()` to define custom REST API endpoints, it is crucial to include a proper `permission_callback`.

🔒 This callback function ensures that only authorized users can access or modify data through your endpoint.

Code example, checking that the user can change options:

```
register_rest_route( 'ebq-seo/v1', '/my-endpoint', array(
    'methods' => 'GET',
    'callback' => 'ebq-seo_callback_function',
    'permission_callback' => function() {
        return current_user_can( 'manage_options' );
    }
) );
```

Please check the register_rest_route() documentation and the current_user_can() documentation.

✅ **When a `permission_callback` is NOT Required**:

There are valid use cases for public endpoints, such as publicly available data (e.g., posts, public metadata) or endpoints designed for unauthenticated access (e.g., fetching public stats or information).

In these cases, you should use `__return_true` as the `permission_callback` to indicate that the endpoint is intentionally public.

🔒 **When a `permission_callback` IS Required**:

For endpoints that involve sensitive data or actions (e.g., getting not public data, creating, updating, or deleting content).

In these cases, you should always implement proper permission checks.

Possible cases found on this plugin's code:

```
includes/class-ebq-rest-proxy.php:314 register_rest_route('ebq/v1', '/entity-coverage/(?P<id>\\d+)', ['methods' => 'GET', 'permission_callback' => [$this, 'can_edit'], 'callback' => [$this, 'entity_coverage'], 'args' => ['id' => ['validate_callback' => static fn($v): bool => is_numeric($v) && (int) $v > 0]]]);
# ↳ Detected: can_edit
# ✨ Uses can_edit(), which only checks edit_posts, but this endpoint operates on a specific post ID and should enforce edit_post for that post before exposing its entity-coverage analysis
```

High position for your admin dashboard menu item

Your plugin registers its top-level admin menu item using a very high menu position, causing it to appear above or alongside core WordPress items.

WordPress has an **established admin hierarchy that users rely on for consistency and navigation**: Posts, Media, Pages, Appearance, Plugins, Users, Tools, Settings, etc. Placing a plugin in a prominent or unexpected position can disrupt this structure and create a cluttered experience, especially on sites with multiple plugins.

Plugins should integrate into WordPress, not compete with core items or other plugins for visibility.

Best practices for menu placement include:

- Add configuration pages under **Settings** - `add_options_page()`
- Add utility functionality under **Tools** - `add_management_page()`
- Add extensions or integrations under the relevant parent plugin (for example, WooCommerce-related pages under WooCommerce) - `add_submenu_page()`
- Use a lower menu position if a top-level item is necessary.

Positioning a menu item prominently is often perceived as a visibility tactic rather than a usability-driven decision. **The WordPress admin interface is a workspace.** Please adjust the menu placement accordingly.

Example(s) from your plugin:

```
includes/class-ebq-hq-page.php:135 add_menu_page(__('EBQ Head Quarter', 'ebq-seo'), __('EBQ HQ', 'ebq-seo'), 'manage_options', self::SLUG, [$this, 'render'], $this->menu_icon(), 3);
```

👉 Your next steps

This is your checklist:

1. Have you read the guidelines and this plugin complies with them?
2. Have you checked for common technical issues?
3. Other details

If there is something that needs to be fixed, please take your time, fix it and update your plugin files at the "Add your plugin" page, while being logged in with your account "malihaider55123". Then, reply to this email.
Please be concise and do not list the changes done — we will review the entire plugin again — but do share any clarifications or important context you want us to know.

If after checking the list and do the changes you feel that everything is right or need further clarification, please reply to this email and a volunteer will assist you.

If you believe there is a requirement you cannot accomplish and choose not to make changes, your plugin submission will be rejected after three months.

Thanks!

By taking these steps, you're helping the Plugin Review Team work more efficiently — meaning your plugin (along with the thousands of others in the queue) can be reviewed faster. 🚀 We really appreciate your contribution!

Disclaimers

If, at any time during the review process, you wish to change your permalink (aka the plugin slug) "ebq-seo", you **must explicitly and clearly tell us what you would like it to be.** Just changing it in your code and in the display name is not sufficient. Remember, permalinks cannot be altered after approval.
This email was partially auto-generated, so please be aware that some information might not be entirely accurate. No personal data was shared with the AI during this process. If you notice any obvious errors or something seems off, feel free to reply — we'll be happy to take a closer look and readjust this automation.

Review ID: AUTOPREREVIEW ❗ LIC ebq-seo/malihaider55123/20May26/T1 20May26/4.0 (P0TDX315650HGN)

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/